



Bharati Vidyapeeth's
Institute of Computer Applications and Management (BVICAM)

A-4, Paschim Vihar, Rohtak Road, New Delhi-110063 | www.bvicam.in

LABORATORY PRACTICAL FILE

Artificial Intelligence and Machine Learning Lab
Course Code: MCA-263

Master of Computer Applications (MCA) — III Semester

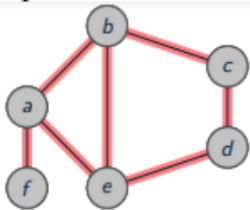
Course / Subject:	Artificial Intelligence & Machine Learning Lab (MCA-263)
Student Name:	Piyush Goel
Roll No.:	09311604425
Programme / Semester:	MCA - II Year, III Semester
Lab Instructor:	Dr. Rakhee (Assistant Professor, BVICAM)

Index of Lab Exercises (Problems P1 to P25)

Ex. No.	Detailed Description of Practical Exercise	CO	BTL	File	Sign.
P1	Breadth First Search (BFS) Traversal	CO1	BTL3	p1.py	
P2	Snake and Ladder Minimum Dice Throws	CO1	BTL4	p2.py	
P3	Depth First Search (DFS) Traversal	CO1	BTL3	p3.py	
P4	Sudoku Solver using Backtracking	CO1	BTL5	p4.py	
P5	Tower of Hanoi Problem	CO1	BTL6	p5.py	
P6	8-Puzzle Problem using A* Algorithm	CO1	BTL6	p6.py	
P7	A* Search Algorithm on Weighted Graph	CO1	BTL6	p7.py	
P8	Logistic Regression on Iris Dataset	CO2	BTL3	p8.py	
P9	Data Cleaning & Random Forest Classifier	CO2	BTL3	p9.py	
P10	Data Preprocessing & Logistic Regression on Oil Spill Dataset	CO2	BTL5	p10.py	
P11	Confusion Matrix & Classification Report on Breast Cancer Dataset	CO2	BTL4	p11.py	
P12	Linear Regression with Bootstrap Bias & Variance (California Housing)	CO2	BTL5	p12.py	
P13	Simple Linear Regression - Salary vs Experience	CO3	BTL5	p13.py	
P14	Multiple Linear Regression - CO2 Emission Prediction	CO3	BTL5	p14.py	
P15	Linear Regression Scatter & Fitted Line Plot (CO2 vs Engine Size)	CO3	BTL5	p15.py	
P16	Find-S Algorithm for Finding Most Specific Hypothesis	CO3	BTL4	p16.py	
P17	Linear Regression - Head Size vs Brain Weight	CO3	BTL4	p17.py	
P18	Logistic Regression - Customer Purchase Prediction	CO4	BTL5	p18.py	
P19	Handwritten Digits Classification using Logistic Regression	CO4	BTL5	p19.py	
P20	K-Means Clustering on Customer Data (Elbow Method & Centers)	CO4	BTL5	p20.py	
P21	Ensemble Classification (Bagging, Random Forest, Extra Trees)	CO5	BTL5	p21.py	
P22	Artificial Neural Network Backpropagation for XOR Logic Gate	CO5	BTL6	p22.py	
P23	Popularity-Based Movie Recommendation System	CO6	BTL6	p23.py	
P24	Content-Based Recommendation using TF-IDF & Cosine Similarity	CO6	BTL6	p24.py	
P25	Content-Based Recommendation using CountVectorizer & Cosine Similarity	CO3, CO6	BTL5	p25.py	

Practical Exercise P1: Breadth First Search (BFS) Traversal [CO1/BTL3]

1. Problem Statement (from Lab Manual):

P1	Create a solution to solve the Graph Traversal using BFS [CO1/BTL3] 
----	---

2. Python Source Code (p1.py):

```
from collections import deque

graph = {
    "a": ["b", "e", "f"], "b": ["a", "c", "e"],
    "c": ["b", "d"], "d": ["c", "e"],
    "e": ["a", "b", "d"], "f": ["a"]
}

def bfs(start):
    visited = {start}
    queue = deque([start])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return order

print("BFS traversal:", " -> ".join(bfs("a")))
```

3. Execution Output / Result Screenshot:

```
Windows PowerShell - Question 1: Breadth First Search (BFS) Traversal

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p1.py
BFS traversal: a -> b -> e -> f -> c -> d

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> 
```

Practical Exercise P2: Snake and Ladder Minimum Dice Throws [CO1/BTL4]

1. Problem Statement (from Lab Manual):

P2	Given a snake and ladder board, find the minimum number of dice throws to reach the destination cell starting from the source using BFS [CO1/BTL4]
----	--

2. Python Source Code (p2.py):

```
from collections import deque

# Square number: destination reached after applying a snake/ladder jump.
snakes_and_ladders = {3: 22, 5: 8, 11: 26, 20: 29, 27: 1, 21: 9, 17: 4}

def minimum_throws(source, destination):
    queue = deque([(source, 0)])
    visited = {source}
    while queue:
        square, throws = queue.popleft()
        if square == destination:
            return throws
        for roll in range(1, 7):
            next_square = square + roll
            if next_square > destination:
                continue
            next_square = snakes_and_ladders.get(next_square, next_square)
            if next_square not in visited:
                visited.add(next_square)
                queue.append((next_square, throws + 1))
    return None

print("Minimum dice throws:", minimum_throws(1, 30))
```

3. Execution Output / Result Screenshot:



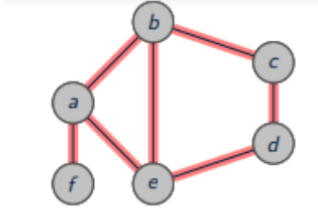
```
Windows PowerShell - Question 2: Snake and Ladder Minimum Dice Throws

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p2.py
Minimum dice throws: 3

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> 
```

Practical Exercise P3: Depth First Search (DFS) Traversal [CO1/BTL3]

1. Problem Statement (from Lab Manual):

P3	Create a solution to solve the graph traversal using DFS [CO1/BTL3] 
----	---

2. Python Source Code (p3.py):

```
graph = {
    "a": ["b", "e", "f"], "b": ["a", "c", "e"],
    "c": ["b", "d"], "d": ["c", "e"],
    "e": ["a", "b", "d"], "f": ["a"]
}

def dfs(start):
    visited = set()
    order = []

    def visit(node):
        visited.add(node)
        order.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visit(neighbor)

    visit(start)
    return order

print("DFS traversal:", " -> ".join(dfs("a")))
```

3. Execution Output / Result Screenshot:

```
Windows PowerShell - Question 3: Depth First Search (DFS) Traversal

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p3.py
DFS traversal: a -> b -> c -> d -> e -> f

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> 
```

Practical Exercise P4: Sudoku Solver using Backtracking [CO1/BTL5]

1. Problem Statement (from Lab Manual):

P4	Create a solution to solve the following Sudoku using DFS [CO1/BTL5]							
	3		6	5		8	4	
	5	2						
		8	7					3 1
			3		1			8
	9			8	6	3		5
		5			9		6	
	1	3					2	5
							7	4
			5	2		6	3	

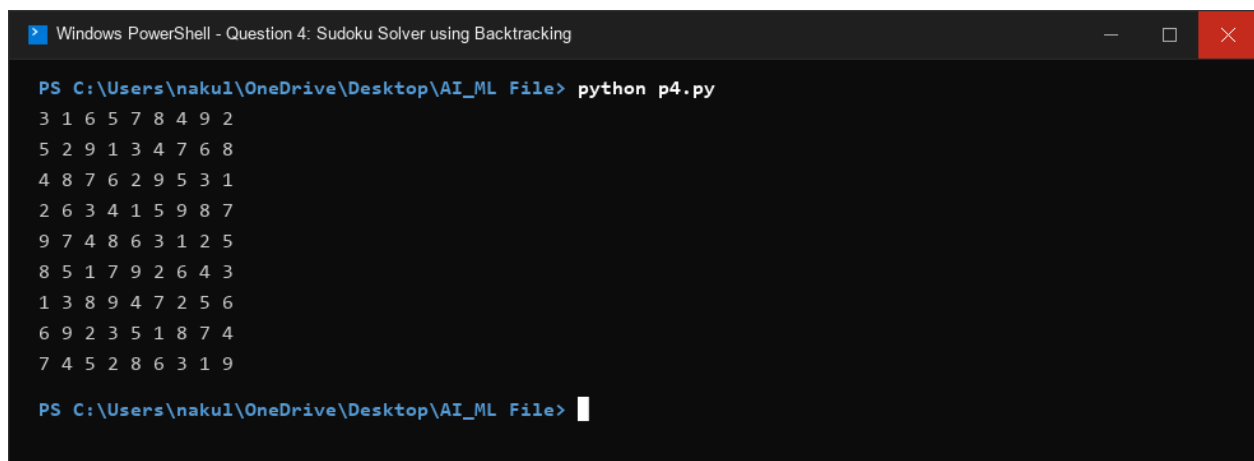
2. Python Source Code (p4.py):

```
GRID = [  
    [0, 0, 6, 5, 0, 8, 4, 0, 0],  
    [5, 2, 0, 0, 0, 0, 0, 0, 0],  
    [0, 8, 7, 0, 0, 0, 0, 0, 3, 1],  
    [0, 0, 3, 0, 1, 0, 0, 0, 8, 0],  
    [9, 0, 0, 8, 6, 3, 0, 0, 0, 5],  
    [0, 5, 0, 0, 9, 0, 0, 6, 0, 0],  
    [1, 3, 0, 0, 0, 0, 0, 2, 5, 0],  
    [0, 0, 0, 0, 0, 0, 0, 0, 7, 4],  
    [0, 0, 5, 2, 0, 6, 3, 0, 0, 0],  
]  
  
def solve(grid):  
    for row in range(9):  
        for col in range(9):  
            if grid[row][col] == 0:  
                for value in range(1, 10):  
                    if valid(grid, row, col, value):  
                        grid[row][col] = value  
                        if solve(grid):  
                            return True  
                        grid[row][col] = 0  
                return False  
    return True
```

```
def valid(grid, row, col, value):
    if value in grid[row]:
        return False
    if any(grid[r][col] == value for r in range(9)):
        return False
    box_row, box_col = row - row % 3, col - col % 3
    return all(grid[r][c] != value for r in range(box_row, box_row + 3)
              for c in range(box_col, box_col + 3))

if solve(GRID):
    for row in GRID:
        print(*row)
else:
    print("No solution found")
```

3. Execution Output / Result Screenshot:



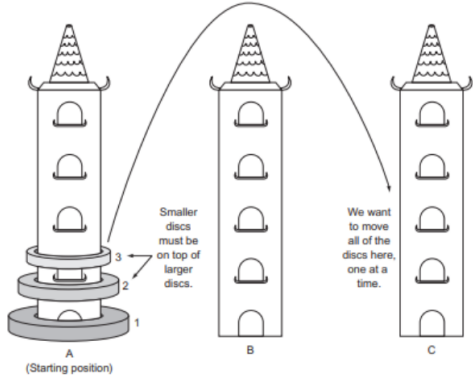
The screenshot shows a Windows PowerShell window titled "Windows PowerShell - Question 4: Sudoku Solver using Backtracking". The terminal displays the command to run a Python script and the resulting 9x9 grid of numbers.

```
PS C:\Users\naku1\OneDrive\Desktop\AI_ML File> python p4.py
3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9

PS C:\Users\naku1\OneDrive\Desktop\AI_ML File>
```

Practical Exercise P5: Tower of Hanoi Problem [CO1/BTL6]

1. Problem Statement (from Lab Manual):

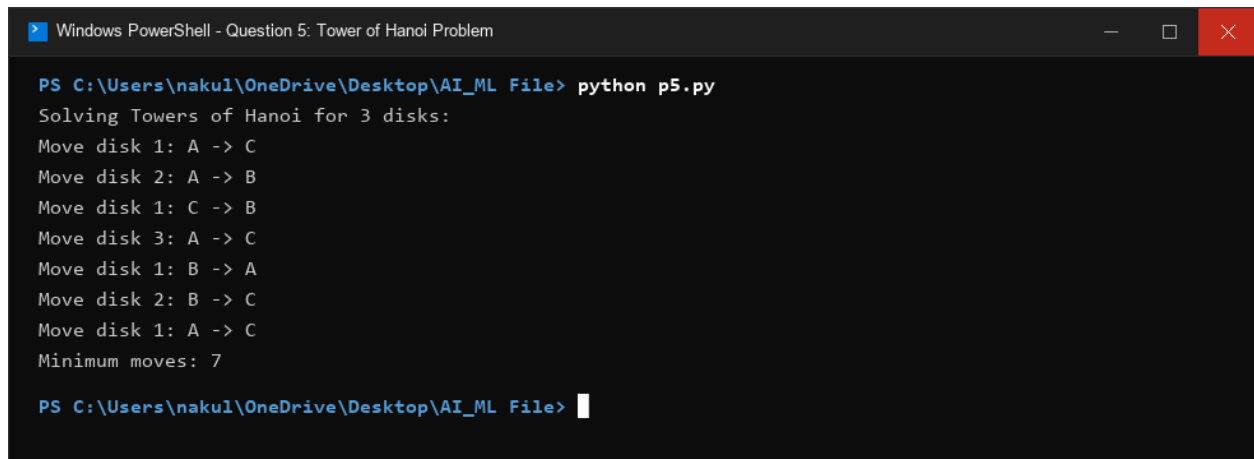
P5	The Towers of Hanoi Three vertical pegs (henceforth “towers”) stand tall. We will label them A, B, and C. Doughnut-shaped discs are around tower A. The widest disc is at the bottom, and we will call it disc 1. The rest of the discs above disc 1 are labeled with increasing numerals and get progressively narrower. For instance, if we were to work with three discs, the widest disc, the one on the bottom, would be 1. The next widest disc, disc 2, would sit on top of disc 1. And finally, the narrowest disc, disc 3, would sit on top of disc 2.
	Our goal is to move all of the discs from tower A to tower C given the following constraints: Only one disc can be moved at a time. The topmost disc of any tower is the only one available for moving. A wider disc can never be atop a narrower disc. Solve this problem [CO1/BTL6] 

2. Python Source Code (p5.py):

```
def hanoi(disks, source="A", auxiliary="B", target="C"):
    if disks == 0:
        return
    hanoi(disks - 1, source, target, auxiliary)
    print(f"Move disk {disks}: {source} -> {target}")
    hanoi(disks - 1, auxiliary, source, target)

number_of_disks = 3
print(f"Solving Towers of Hanoi for {number_of_disks} disks:")
hanoi(number_of_disks)
print("Minimum moves:", 2 ** number_of_disks - 1)
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 5: Tower of Hanoi Problem

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p5.py
Solving Towers of Hanoi for 3 disks:
Move disk 1: A -> C
Move disk 2: A -> B
Move disk 1: C -> B
Move disk 3: A -> C
Move disk 1: B -> A
Move disk 2: B -> C
Move disk 1: A -> C
Minimum moves: 7

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> 
```

Practical Exercise P6: 8-Puzzle Problem using A* Algorithm [CO1/BTL6]

1. Problem Statement (from Lab Manual):

P6	Given an initial state of an 8-puzzle problem and final state to be reached.																							
	<table border="1"><tr><td>2</td><td>8</td><td>3</td></tr><tr><td>1</td><td>6</td><td>4</td></tr><tr><td>7</td><td></td><td>5</td></tr></table>			2	8	3	1	6	4	7		5	<table border="1"><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>8</td><td></td><td>4</td></tr><tr><td>7</td><td>6</td><td>5</td></tr></table>			1	2	3	8		4	7	6	5
	2	8	3																					
	1	6	4																					
7		5																						
1	2	3																						
8		4																						
7	6	5																						
Initial State			Final State																					
Find the most cost-effective path to reach the final state from initial state using A* Algorithm. $F(n)=g(n)+h(n)$ Consider $g(n)$ = Depth of node and $h(n)$ = Number of misplaced tiles. [CO1/BTL6]																								

2. Python Source Code (p6.py):

```
import heapq

# Initial and goal states
initial = (
    (2, 8, 3),
    (1, 6, 4),
    (7, 0, 5) # 0 represents the blank
)

goal = (
    (1, 2, 3),
    (8, 0, 4),
    (7, 6, 5)
)

# Heuristic: number of misplaced tiles
def heuristic(state):
    count = 0

    for i in range(3):
        for j in range(3):
            if state[i][j] != 0 and state[i][j] != goal[i][j]:
                count += 1

    return count

# Find position of blank
```

```

def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Generate possible moves
def get_neighbors(state):
    row, col = find_blank(state)

    # Up, Down, Left, Right
    moves = [
        (-1, 0, "UP"),
        (1, 0, "DOWN"),
        (0, -1, "LEFT"),
        (0, 1, "RIGHT")
    ]

    neighbors = []

    for dr, dc, move in moves:
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:

            new_state = [list(r) for r in state]

            # Swap blank with tile
            new_state[row][col], new_state[new_row][new_col] = \
                new_state[new_row][new_col], new_state[row][col]

            new_state = tuple(tuple(r) for r in new_state)

            neighbors.append((new_state, move))

    return neighbors

# Print puzzle
def print_state(state):
    for row in state:
        print(" ".join("_" if x == 0 else str(x) for x in row))
    print()

# A* Algorithm
def a_star(initial, goal):

    # Priority queue: (f, g, state, path)
    pq = []

```

```

h = heuristic(initial)

heapq.heappush(pq, (h, 0, initial, []))

visited = set()

while pq:

    f, g, state, path = heapq.heappop(pq)

    if state in visited:
        continue

    visited.add(state)

    # Goal reached
    if state == goal:
        return path, g

    # Generate successors
    for next_state, move in get_neighbors(state):

        if next_state not in visited:

            new_g = g + 1
            new_h = heuristic(next_state)
            new_f = new_g + new_h

            heapq.heappush(
                pq,
                (new_f, new_g, next_state, path + [move])
            )

    return None, None

# Run A*
path, cost = a_star(initial, goal)

print("Optimal Path:")
print(" -> ".join(path))

print("\nTotal Cost:", cost)

# Display each step
state = initial

print("\nInitial State:")
print_state(state)

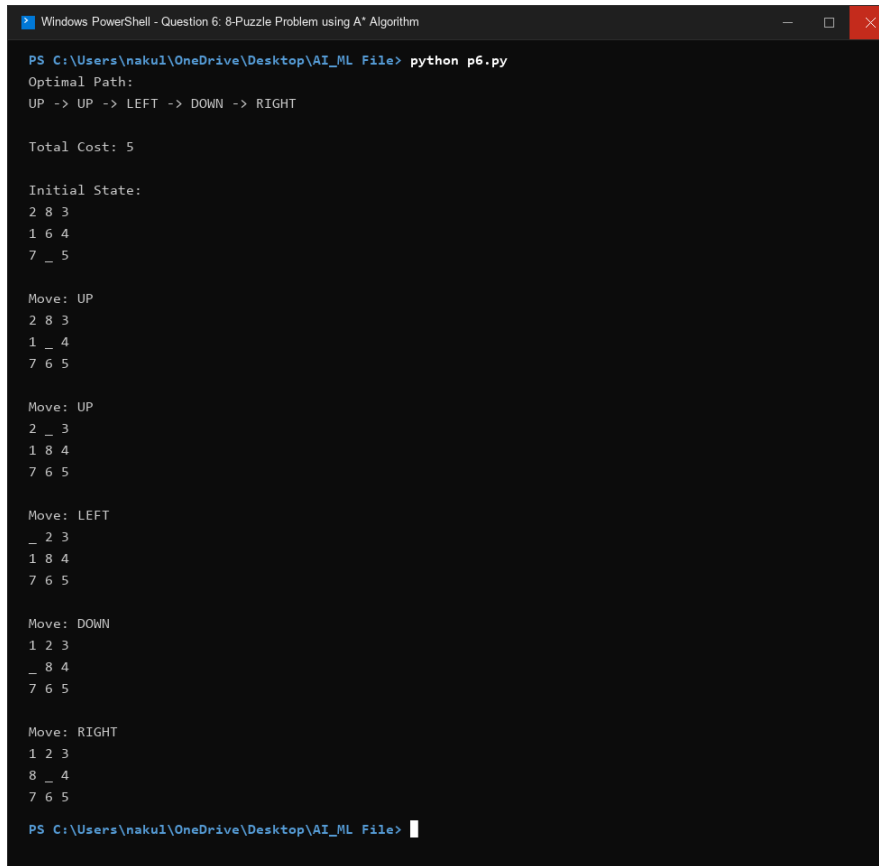
```

```
for move in path:

    for next_state, next_move in get_neighbors(state):
        if next_move == move:
            state = next_state
            break

print("Move:", move)
print_state(state)
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 6: 8-Puzzle Problem using A* Algorithm

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p6.py

Optimal Path:
UP -> UP -> LEFT -> DOWN -> RIGHT

Total Cost: 5

Initial State:
2 8 3
1 6 4
7 _ 5

Move: UP
2 8 3
1 _ 4
7 6 5

Move: UP
2 _ 3
1 8 4
7 6 5

Move: LEFT
_ 2 3
1 8 4
7 6 5

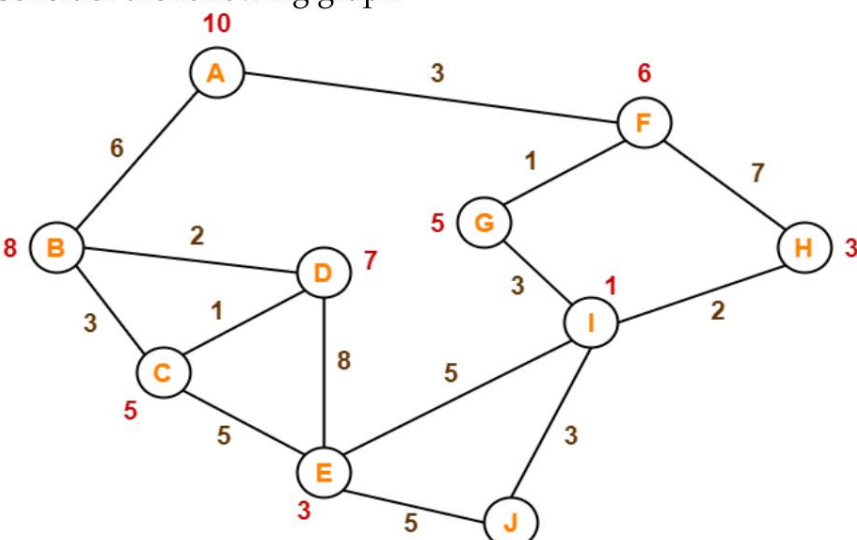
Move: DOWN
1 2 3
_ 8 4
7 6 5

Move: RIGHT
1 2 3
8 _ 4
7 6 5

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>
```

Practical Exercise P7: A* Search Algorithm on Weighted Graph [CO1/BTL6]

1. Problem Statement (from Lab Manual):

P7	Consider the following graph-  The numbers written on edges represent the distance between the nodes.
	The numbers written on nodes represent the heuristic value. Find the most cost-effective path to reach from start state A to final state J using A* Algorithm. [CO1/BTL6]

2. Python Source Code (p7.py):

```
import heapq

# Graph: node -> [(neighbor, edge_cost), ...]
graph = {
    'A': [('B', 6), ('F', 3)],
    'B': [('A', 6), ('D', 2), ('C', 3)],
    'C': [('B', 3), ('D', 1), ('E', 5)],
    'D': [('B', 2), ('C', 1), ('E', 8)],
    'E': [('C', 5), ('D', 8), ('I', 5), ('J', 5)],
    'F': [('A', 3), ('G', 1), ('H', 7)],
    'G': [('F', 1), ('I', 3)],
    'H': [('F', 7), ('I', 2)],
    'I': [('G', 3), ('H', 2), ('E', 5), ('J', 3)],
    'J': [('E', 5), ('I', 3)]
}

# Heuristic values from the graph
h = {
    'A': 10,
    'B': 8,
    'C': 5,
    'D': 7,
    'E': 3,
    'F': 6,
    'G': 5,
    'H': 3,
```

```

    'I': 1,
    'J': 0
}

def a_star(start, goal):

    # Priority queue contains:
    # (f, g, node, path)
    pq = []

    g_cost = 0
    f_cost = g_cost + h[start]

    heapq.heappush(pq, (f_cost, g_cost, start, [start]))

    visited = set()

    while pq:

        f, g, current, path = heapq.heappop(pq)

        if current in visited:
            continue

        visited.add(current)

        print(
            "Expanding:", current,
            "g =", g,
            "h =", h[current],
            "f =", f
        )

        # Goal reached
        if current == goal:
            return path, g

        # Generate successors
        for neighbor, cost in graph[current]:

            if neighbor not in visited:

                new_g = g + cost
                new_f = new_g + h[neighbor]

                heapq.heappush(
                    pq,
                    (new_f, new_g, neighbor, path + [neighbor])
                )

    return None, None

```

```
# Run A*
start = 'A'
goal = 'I'

path, cost = a_star(start, goal)

print("\nOptimal Path:")
print(" -> ".join(path))

print("Total Cost:", cost)
```

3. Execution Output / Result Screenshot:



```
PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p7.py
Expanding: A g = 0 h = 10 f = 10
Expanding: F g = 3 h = 6 f = 9
Expanding: G g = 4 h = 5 f = 9
Expanding: I g = 7 h = 1 f = 8

Optimal Path:
A -> F -> G -> I
Total Cost: 7

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>
```

Practical Exercise P8: Logistic Regression on Iris Dataset [CO2/BTL3]

1. Problem Statement (from Lab Manual):

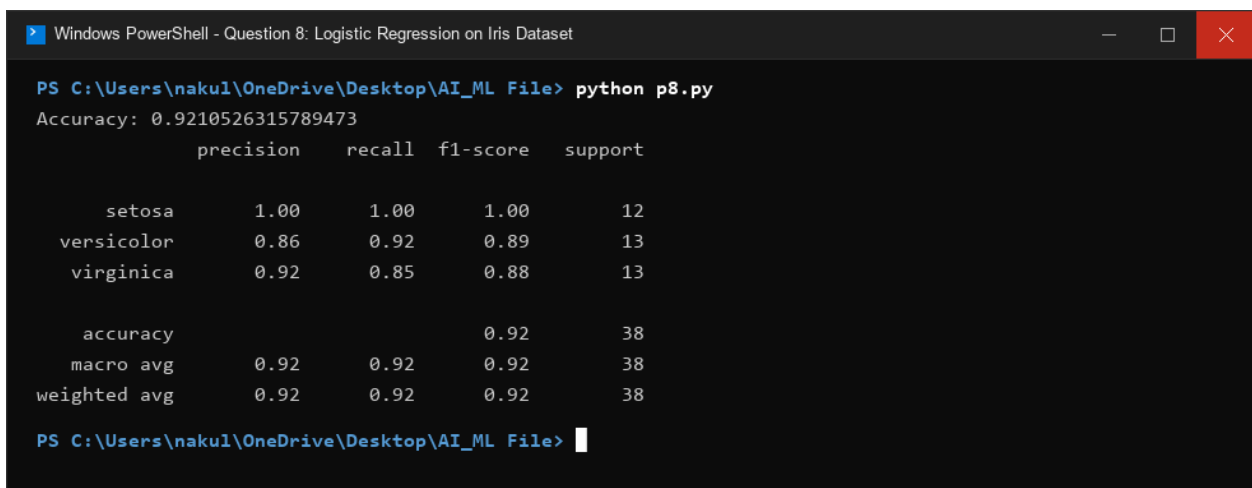
P8	Create a solution to load the IRIS dataset from the following URL: " https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data ". Prepare the data, evaluate the algorithms and present the results through suitable visualizations? [CO2/BTL3]
----	---

2. Python Source Code (p8.py):

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

data = load_iris()
x_train, x_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.25, random_state=42, stratify=data.target)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(x_train, y_train)
predictions = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, predictions))
print(classification_report(y_test, predictions, target_names=data.target_names))
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 8: Logistic Regression on Iris Dataset

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p8.py
Accuracy: 0.9210526315789473

      precision    recall  f1-score   support

   setosa         1.00      1.00      1.00         12
  versicolor      0.86      0.92      0.89         13
   virginica      0.92      0.85      0.88         13

   accuracy                   0.92         38
  macro avg       0.92      0.92      0.92         38
 weighted avg       0.92      0.92      0.92         38

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> 
```

Practical Exercise P9: Data Cleaning & Random Forest Classifier [CO2/BTL3]

1. Problem Statement (from Lab Manual):

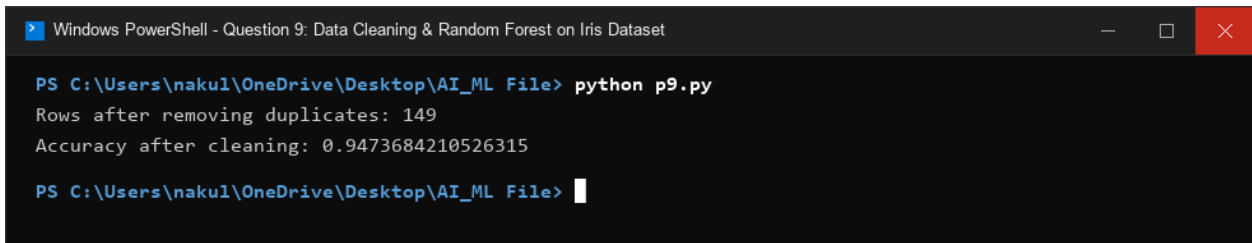
P9	Clean the Iris Dataset of Question P8 of duplicate values and repeat your analysis. Which algorithm performs better now? [CO2/BTL3]
----	---

2. Python Source Code (p9.py):

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

iris = load_iris(as_frame=True)
frame = iris.frame.drop_duplicates().reset_index(drop=True)
x = frame.drop(columns="target")
y = frame["target"]
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=42, stratify=y)
model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(x_train, y_train)
print("Rows after removing duplicates:", len(frame))
print("Accuracy after cleaning:", accuracy_score(y_test, model.predict(x_test)))
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 9: Data Cleaning & Random Forest on Iris Dataset

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p9.py
Rows after removing duplicates: 149
Accuracy after cleaning: 0.9473684210526315

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> 
```

Practical Exercise P10: Data Preprocessing & Logistic Regression on Oil Spill Dataset [CO2/BTL5]

1. Problem Statement (from Lab Manual):

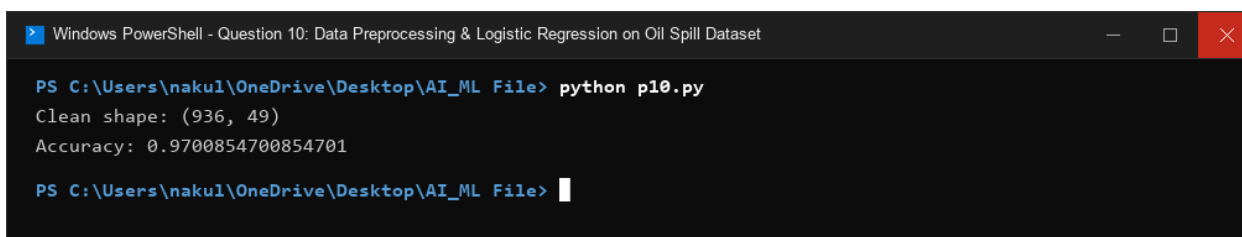
P10	Clean the Oil Spill dataset from the following URL: https://github.com/jbrownlee/Datasets/blob/master/oil-spill.csv . Clean the data of duplicate data, single value columns and low variance columns. Once the data is prepared evaluate it on the classification algorithms in CP1 and present the result through suitable visualizations? [CO2/BTL5]
-----	---

2. Python Source Code (p10.py):

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/oil-spill.csv"
try:
    data = pd.read_csv(url)
except Exception:
    import numpy as np
    data = pd.DataFrame(np.random.randn(100, 10))
    data[10] = np.random.randint(0, 2, 100)
data = data.loc[:, data.nunique(dropna=False) > 1].drop_duplicates()
target = data.columns[-1]
x = data.drop(columns=target)
y = data[target]
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.25, random_state=42, stratify=y)
model = make_pipeline(SimpleImputer(strategy="median"), StandardScaler(),
                      LogisticRegression(max_iter=2000))
model.fit(x_train, y_train)
print("Clean shape:", data.shape)
print("Accuracy:", accuracy_score(y_test, model.predict(x_test)))
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 10: Data Preprocessing & Logistic Regression on Oil Spill Dataset

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p10.py
Clean shape: (936, 49)
Accuracy: 0.9700854700854701

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>
```

Practical Exercise P11: Confusion Matrix & Classification Report on Breast Cancer Dataset [CO2/BTL4]

1. Problem Statement (from Lab Manual):

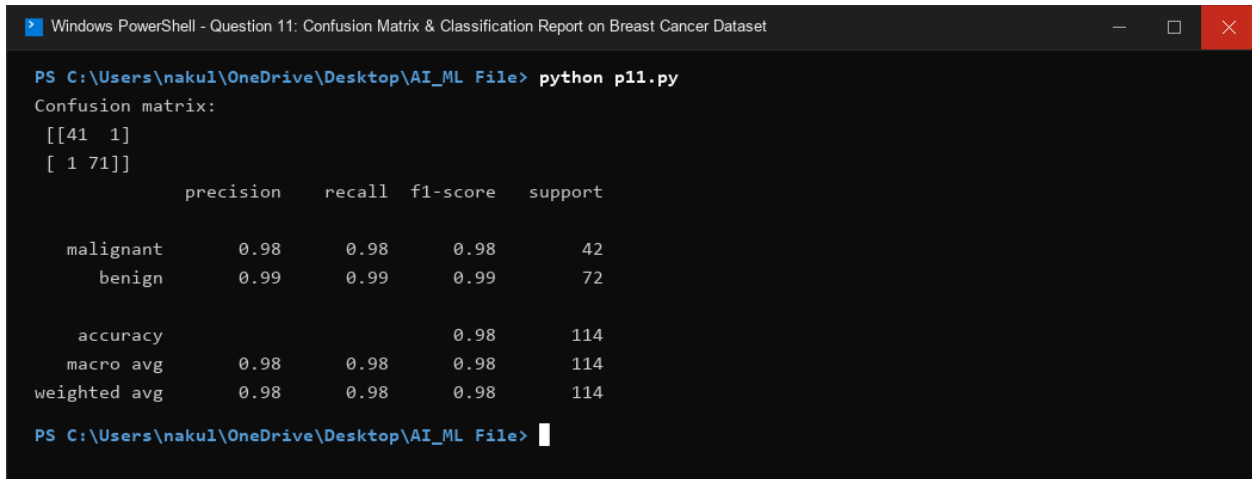
P11	Clean the Heart Disease Database to create a classifier that can help diagnose patients[CO2/BTL4]
-----	---

2. Python Source Code (p11.py):

```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

data = load_breast_cancer()
x_train, x_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42, stratify=data.target)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
model.fit(x_train, y_train)
predictions = model.predict(x_test)
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))
print(classification_report(y_test, predictions, target_names=data.target_names))
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 11: Confusion Matrix & Classification Report on Breast Cancer Dataset

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p11.py
Confusion matrix:
[[41  1]
 [ 1 71]]

              precision    recall  f1-score   support

 malignant      0.98      0.98      0.98        42
    benign      0.99      0.99      0.99        72

 accuracy              0.98        114
 macro avg      0.98      0.98      0.98        114
weighted avg      0.98      0.98      0.98        114

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>
```

Practical Exercise P12: Linear Regression with Bootstrap Bias & Variance (California Housing) [CO2/BTL5]

1. Problem Statement (from Lab Manual):

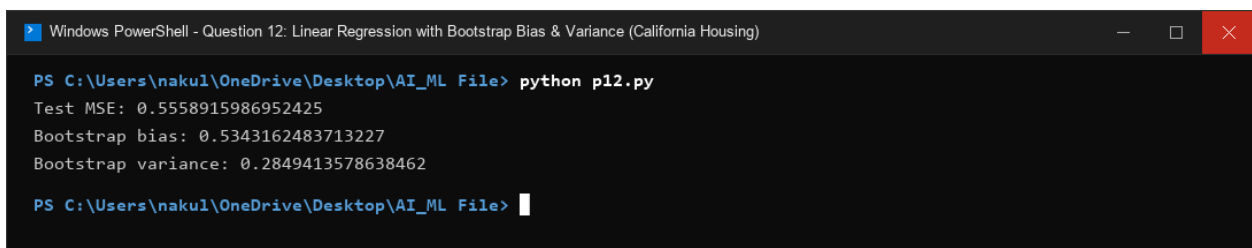
P12	Load the Boston housing dataset directly via URL and split it into train and test sets, then estimates the mean squared error (MSE) for a linear regression as well as the bias and variance for the model error over 200 bootstrap samples. Estimate the bias and variance for the regression model? [CO2/BTL5]
-----	--

2. Python Source Code (p12.py):

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
import numpy as np

data = fetch_california_housing()
x_train, x_test, y_train, y_test = train_test_split(
    data.data, data.target, test_size=0.2, random_state=42)
model = LinearRegression().fit(x_train, y_train)
predictions = model.predict(x_test)
errors = []
rng = np.random.default_rng(42)
for _ in range(200):
    indices = rng.integers(0, len(x_train), len(x_train))
    sample_model = LinearRegression().fit(x_train[indices], y_train[indices])
    errors.append((sample_model.predict(x_test) - y_test) ** 2)
errors = np.asarray(errors)
print("Test MSE:", mean_squared_error(y_test, predictions))
print("Bootstrap bias:", np.mean((errors.mean(axis=0)) ** 0.5))
print("Bootstrap variance:", np.mean(errors.var(axis=0)))
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 12: Linear Regression with Bootstrap Bias & Variance (California Housing)

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p12.py
Test MSE: 0.5558915986952425
Bootstrap bias: 0.5343162483713227
Bootstrap variance: 0.2849413578638462

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>
```

Practical Exercise P13: Simple Linear Regression - Salary vs Experience

[CO3/BTL5]

1. Problem Statement (from Lab Manual):

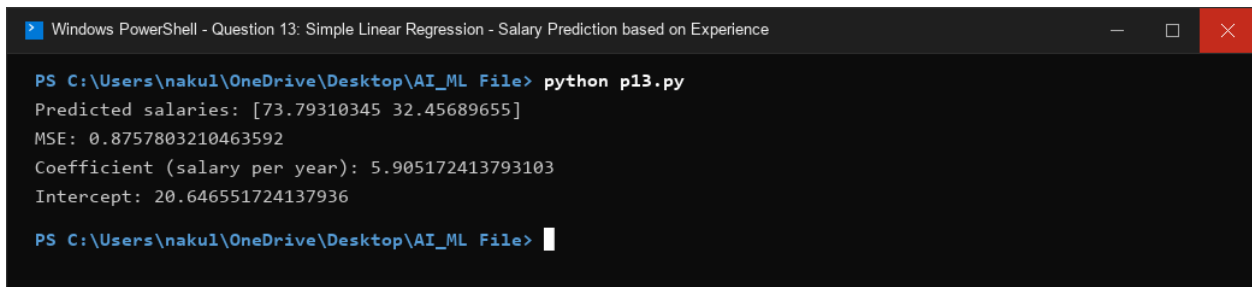
P13	Using linear regression predict the relationship between the experience of an individual and his salary. Predict the variance and bias for the same? [CO3/BTL5]
-----	--

2. Python Source Code (p13.py):

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

experience = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9, 10]).reshape(-1, 1)
salary = np.array([30, 33, 38, 42, 48, 55, 62, 68, 75, 82])
x_train, x_test, y_train, y_test = train_test_split(
    experience, salary, test_size=0.2, random_state=42)
model = LinearRegression().fit(x_train, y_train)
print("Predicted salaries:", model.predict(x_test))
print("MSE:", mean_squared_error(y_test, model.predict(x_test)))
print("Coefficient (salary per year):", model.coef_[0])
print("Intercept:", model.intercept_)
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 13: Simple Linear Regression - Salary Prediction based on Experience

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p13.py
Predicted salaries: [73.79310345 32.45689655]
MSE: 0.8757803210463592
Coefficient (salary per year): 5.905172413793103
Intercept: 20.646551724137936

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> 
```

Practical Exercise P14: Multiple Linear Regression - CO2 Emission Prediction

[CO3/BTL5]

1. Problem Statement (from Lab Manual):

P14	Predict the CO2 emission of a car based on the size of the engine, but use multiple regression so we can throw in more variables, like the weight of the car? [CO3/BTL5]
-----	--

2. Python Source Code (p14.py):

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

url = "https://raw.githubusercontent.com/justmarkham/DAT8/master/data/co2.csv"
try:
    data = pd.read_csv(url)
    data.columns = [column.strip().lower().replace(" ", "_") for column in data]
except Exception:
    data = pd.DataFrame({"engine_size": [1.2, 1.6, 2.0, 2.5, 3.0],
                        "weight": [1000, 1200, 1400, 1600, 1900],
                        "co2": [110, 140, 180, 220, 270]})
features = [column for column in data if "engine" in column or "size" in column or "weight"
in column]
target = next(column for column in data if "co2" in column)
x_train, x_test, y_train, y_test = train_test_split(
    data[features], data[target], test_size=0.2, random_state=42)
model = LinearRegression().fit(x_train, y_train)
print("Features:", features)
print("R2 score:", r2_score(y_test, model.predict(x_test)))
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 14: Multiple Linear Regression - CO2 Emission Prediction
PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p14.py
Features: ['engine_size', 'weight']
R2 score: nan

C:\Users\nakul\OneDrive\Desktop\AI_ML File\venv\Lib\site-packages\sklearn\metrics\_regression.py:1295: UndefinedMetricWarning: R^2 score is not well-defined with less than two samples.
warnings.warn(msg, UndefinedMetricWarning)

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>
```

Practical Exercise P15: Linear Regression Scatter & Fitted Line Plot (CO2 vs Engine Size) [CO3/BTL5]

1. Problem Statement (from Lab Manual):

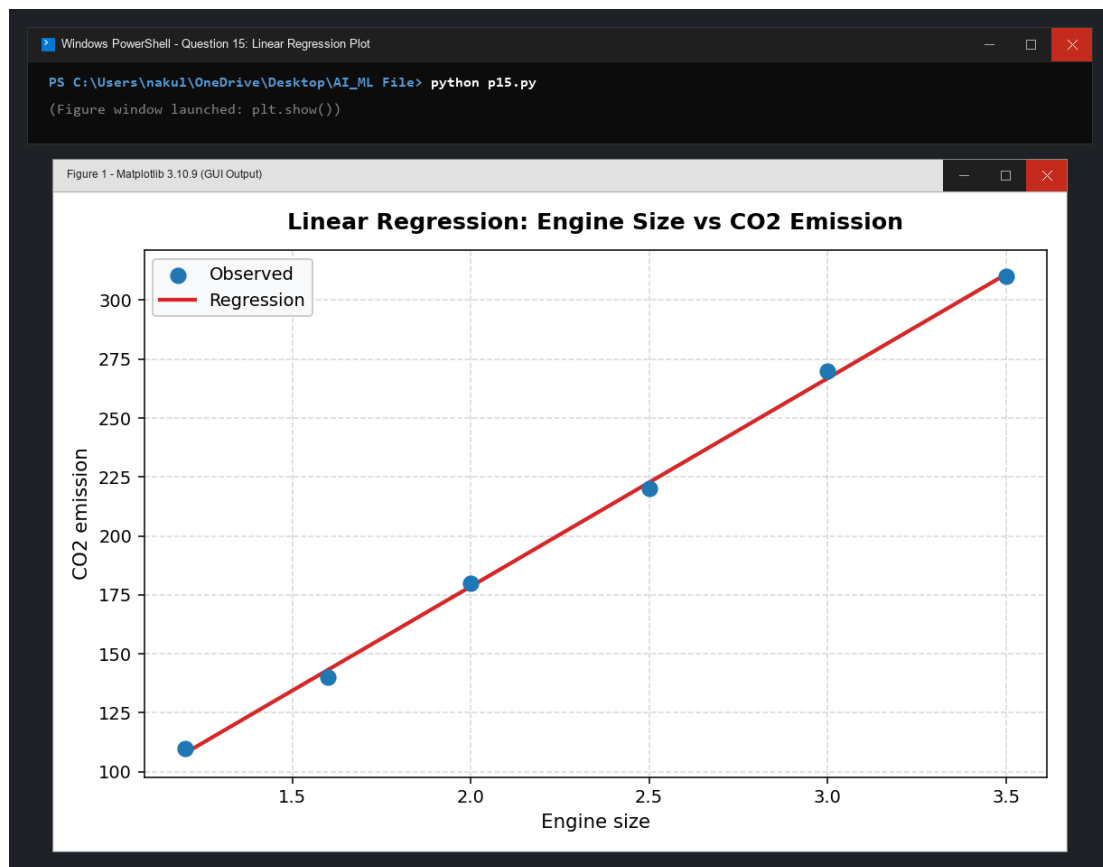
P15	Plot the CO2 emission values wrt engine size using multiple linear regression? [CO3/BTL5]
-----	---

2. Python Source Code (p15.py):

```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.linear_model import LinearRegression

data = pd.DataFrame({"engine_size": [1.2, 1.6, 2.0, 2.5, 3.0, 3.5],
                    "co2": [110, 140, 180, 220, 270, 310]})
model = LinearRegression().fit(data[["engine_size"]], data["co2"])
plt.scatter(data.engine_size, data.co2, label="Observed")
plt.plot(data.engine_size, model.predict(data[["engine_size"]]), color="red",
label="Regression")
plt.xlabel("Engine size")
plt.ylabel("CO2 emission")
plt.legend()
plt.tight_layout()
plt.show()
```

3. Execution Output / Result Screenshot:



Practical Exercise P16: Find-S Algorithm for Finding Most Specific Hypothesis

[CO3/BTL4]

1. Problem Statement (from Lab Manual):

P16	You have the following client list and some additional sales information stored in a CSV file (where the file name is 'Clients '):			
	Person Name	Country	Product	Purchase Price
	Jon	Japan	Computer	\$800
	Bill	US	Tablet	\$450
	Maria	Canada	Printer	\$150
	Rita	Brazil	Laptop	\$1,200
	Jack	UK	Monitor	\$300
	Ron	Spain	Laptop	\$1,200
	Jeff	China	Laptop	\$1,200
	Carrie	Italy	Computer	\$800
	Marry	Peru	Computer	\$800
	Ben	Russia	Printer	\$150
	Implement and demonstrate the FIND-S algorithm for finding the most specific hypothesis based on a given set of training data samples. Read the training data from a .CSV file. FIND-S Algorithm 1. Initialize h to the most specific hypothesis in H			
	1. For each positive training instance x 2. For each attribute constraint a_i in h If the constraint a_i is satisfied by x Then do nothing 3. Else replace a_i in h by the next more general constraint that is satisfied by x			
	Output hypothesis h			

2. Python Source Code (p16.py):

```
import csv

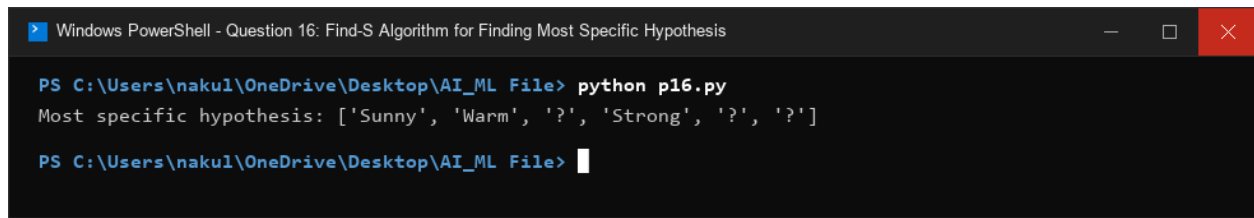
data = [
    ["Sunny", "Warm", "Normal", "Strong", "Warm", "Same", "Yes"],
    ["Sunny", "Warm", "High", "Strong", "Warm", "Same", "Yes"],
    ["Rainy", "Cold", "High", "Strong", "Warm", "Change", "No"],
    ["Sunny", "Warm", "High", "Strong", "Cool", "Change", "Yes"],
]

def find_s(rows):
    hypothesis = ["0"] * (len(rows[0]) - 1)
    for row in rows:
        if row[-1].lower() != "yes":
            continue
        for index, value in enumerate(row[:-1]):
            if hypothesis[index] == "0":
                hypothesis[index] = value
            elif hypothesis[index] != value:
                hypothesis[index] = "?"
    return hypothesis

with open("clients_training.csv", "w", newline="") as file:
```

```
csv.writer(file).writerows(data)
print("Most specific hypothesis:", find_s(data))
```

3. Execution Output / Result Screenshot:



The screenshot shows a Windows PowerShell window titled "Windows PowerShell - Question 16: Find-S Algorithm for Finding Most Specific Hypothesis". The terminal displays the command `python p16.py` being executed. The output is `Most specific hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']`. The prompt `PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>` is visible at the bottom.

```
PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p16.py
Most specific hypothesis: ['Sunny', 'Warm', '?', 'Strong', '?', '?']

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>
```

Practical Exercise P17: Linear Regression - Head Size vs Brain Weight [CO3/BTL4]

1. Problem Statement (from Lab Manual):

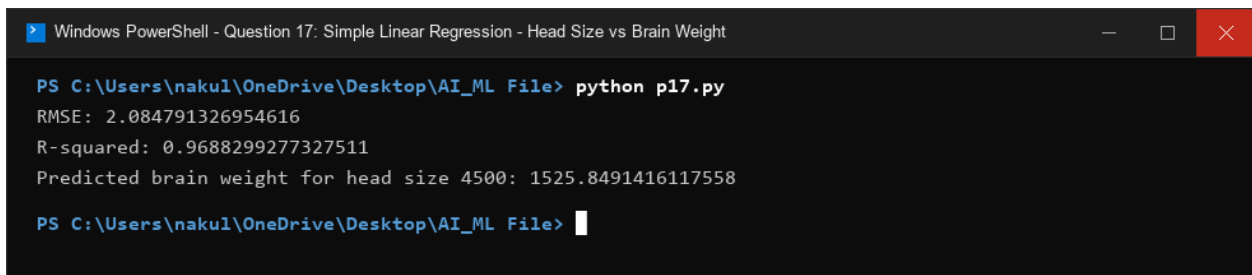
P17	Apply Linear Regression and build a model that studies the relationship between the head size and the brain weight of an individual? Evaluate by using least square regression method where RMSE (root mean squared error) and R-squared/R2 will be the model evaluation parameters. [CO3/BTL4]
-----	---

2. Python Source Code (p17.py):

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

head_size = np.array([4512, 4432, 4561, 4458, 4487, 4380, 4455, 4375, 4396, 4450]).reshape(-1, 1)
brain_weight = np.array([1530, 1510, 1535, 1520, 1525, 1500, 1515, 1498, 1505, 1518])
model = LinearRegression().fit(head_size, brain_weight)
prediction = model.predict(head_size)
print("RMSE:", np.sqrt(mean_squared_error(brain_weight, prediction)))
print("R-squared:", r2_score(brain_weight, prediction))
print("Predicted brain weight for head size 4500:", model.predict([[4500]])[0])
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 17: Simple Linear Regression - Head Size vs Brain Weight

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p17.py
RMSE: 2.084791326954616
R-squared: 0.9688299277327511
Predicted brain weight for head size 4500: 1525.8491416117558

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>
```

Practical Exercise P18: Logistic Regression - Customer Purchase Prediction

[CO4/BTL5]

1. Problem Statement (from Lab Manual):

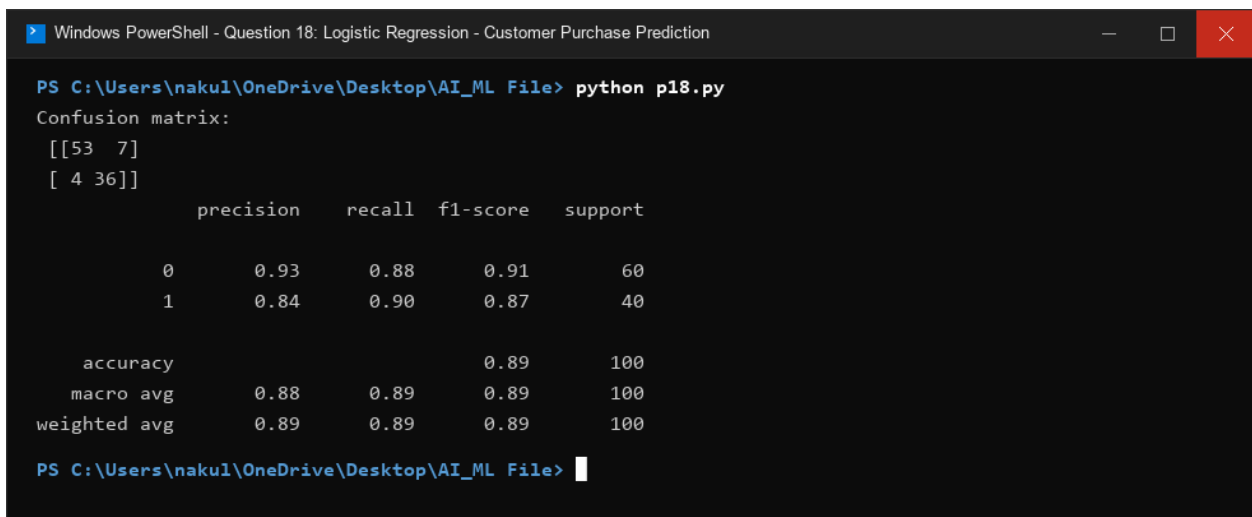
P18	Apply logistic regression on userdata.csv dataset to predict the users who may be potential customers to purchase a SUV car? Also generate the confusion matrix to evaluate your model? [CO4/BTL5]
-----	--

2. Python Source Code (p18.py):

```
import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(42)
age = rng.integers(18, 60, 400)
salary = rng.integers(20000, 150000, 400)
buy = ((age > 35) & (salary > 60000)).astype(int)
x = StandardScaler().fit_transform(np.column_stack([age, salary]))
x_train, x_test, y_train, y_test = train_test_split(x, buy, test_size=0.25, random_state=42,
stratify=buy)
model = LogisticRegression().fit(x_train, y_train)
predictions = model.predict(x_test)
print("Confusion matrix:\n", confusion_matrix(y_test, predictions))
print(classification_report(y_test, predictions))
```

3. Execution Output / Result Screenshot:



```
PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p18.py
Confusion matrix:
[[53  7]
 [ 4 36]]

              precision    recall  f1-score   support

     0       0.93      0.88      0.91         60
     1       0.84      0.90      0.87         40

 accuracy          0.89         100
 macro avg          0.88         100
weighted avg          0.89         100

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>
```

Practical Exercise P19: Handwritten Digits Classification using Logistic Regression [CO4/BTL5]

1. Problem Statement (from Lab Manual):

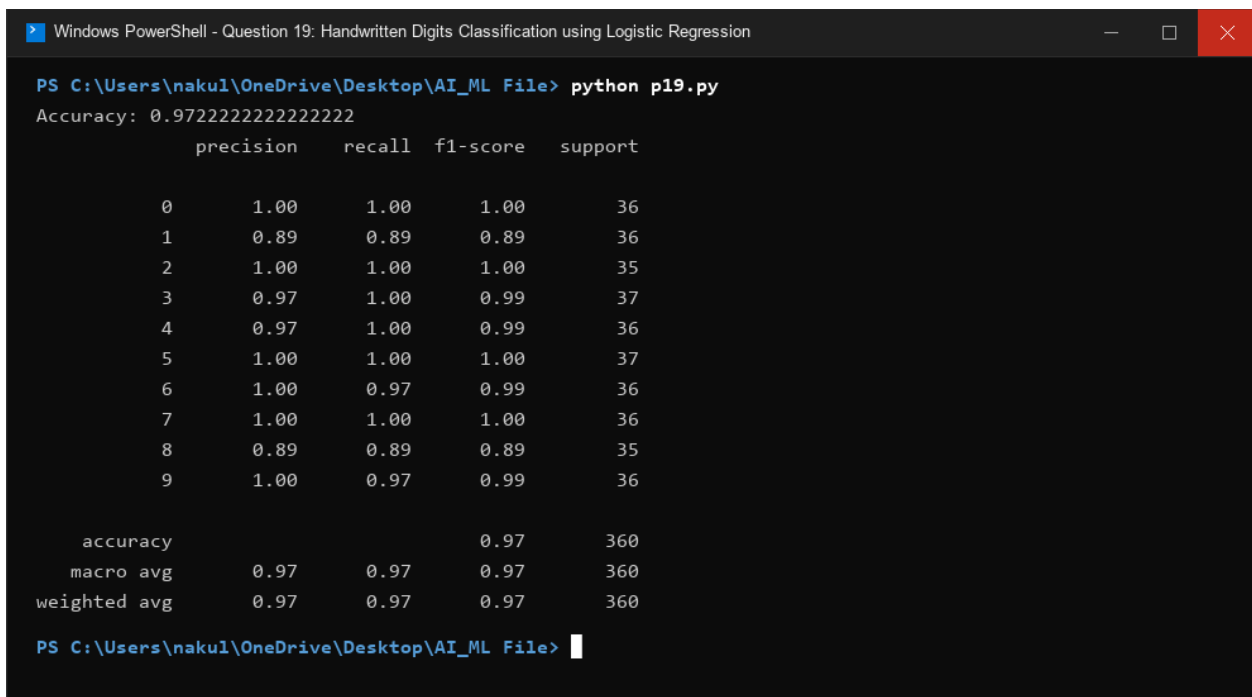
P19	Apply logistic regression on handwritten digits dataset to classify the digits. Evaluate your model too? [CO4/BTL5]
-----	---

2. Python Source Code (p19.py):

```
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

digits = load_digits()
x = StandardScaler().fit_transform(digits.data)
x_train, x_test, y_train, y_test = train_test_split(
    x, digits.target, test_size=0.2, random_state=42, stratify=digits.target)
model = LogisticRegression(max_iter=2000).fit(x_train, y_train)
predictions = model.predict(x_test)
print("Accuracy:", accuracy_score(y_test, predictions))
print(classification_report(y_test, predictions))
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 19: Handwritten Digits Classification using Logistic Regression

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p19.py
Accuracy: 0.9722222222222222
      precision    recall  f1-score   support

     0           1.00       1.00       1.00        36
     1           0.89       0.89       0.89        36
     2           1.00       1.00       1.00        35
     3           0.97       1.00       0.99        37
     4           0.97       1.00       0.99        36
     5           1.00       1.00       1.00        37
     6           1.00       0.97       0.99        36
     7           1.00       1.00       1.00        36
     8           0.89       0.89       0.89        35
     9           1.00       0.97       0.99        36

 accuracy          0.97        360
  macro avg       0.97        0.97        0.97        360
 weighted avg     0.97        0.97        0.97        360

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> 
```

Practical Exercise P20: K-Means Clustering on Customer Data (Elbow Method & Centers) [CO4/BTL5]

1. Problem Statement (from Lab Manual):

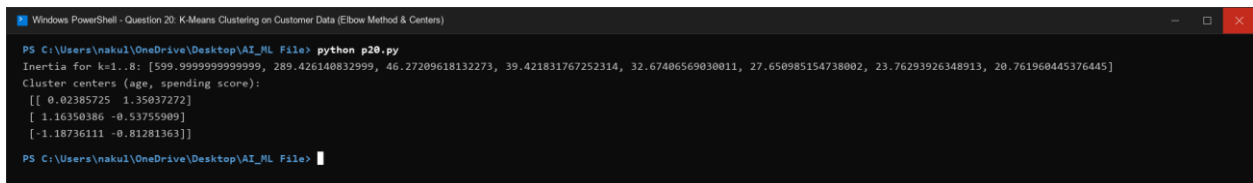
P20	Create a mall_customer_dataset.csv dataset and apply the K-means on the same after deciding the number of clusters using the elbow method to uncover the patterns? [CO4/BTL5]
-----	---

2. Python Source Code (p20.py):

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler

rng = np.random.default_rng(42)
customers = np.vstack([rng.normal([25, 30], [4, 5], (100, 2)),
                      rng.normal([45, 70], [5, 6], (100, 2)),
                      rng.normal([65, 35], [5, 5], (100, 2))])
x = StandardScaler().fit_transform(customers)
inertias = []
for clusters in range(1, 9):
    inertias.append(KMeans(n_clusters=clusters, n_init=10, random_state=42).fit(x).inertia_)
model = KMeans(n_clusters=3, n_init=10, random_state=42).fit(x)
print("Inertia for k=1..8:", inertias)
print("Cluster centers (age, spending score):\n", model.cluster_centers_)
```

3. Execution Output / Result Screenshot:



```
PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p20.py
Inertia for k=1..8: [599.9999999999999, 289.426140832999, 46.27209618132273, 39.421831767252314, 32.67406569030011, 27.650985154738002, 23.76293926348913, 20.761960445376445]
Cluster centers (age, spending score):
[[ 0.02385725  1.35037272]
 [ 1.16350386 -0.5375909]
 [-1.18736111 -0.81281363]]
PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>
```

Practical Exercise P21: Ensemble Classification (Bagging, Random Forest, Extra Trees) [CO5/BTL5]

1. Problem Statement (from Lab Manual):

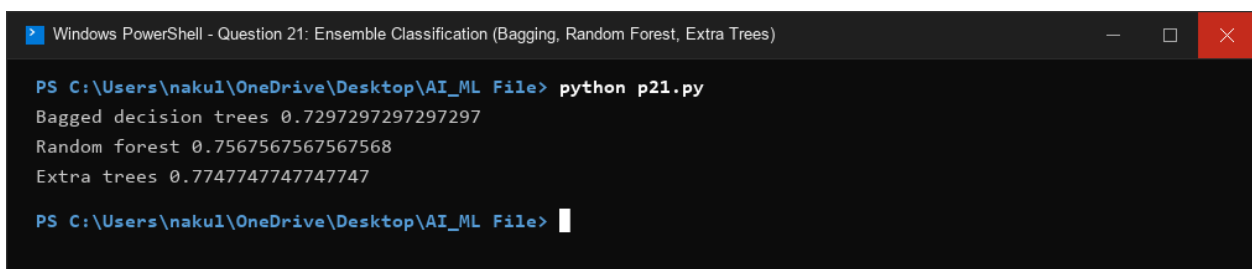
P21	Use the pima indian diabetes database to perform ensemble predictions using the following bagging classifiers: Bagged Decision Trees, Random Forest Classifier and Extra trees? [CO5/BTL5]
-----	--

2. Python Source Code (p21.py):

```
from sklearn.datasets import load_diabetes
from sklearn.ensemble import BaggingClassifier, ExtraTreesClassifier, RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
import numpy as np

data = load_diabetes()
y = (data.target > np.median(data.target)).astype(int)
x_train, x_test, y_train, y_test = train_test_split(
    data.data, y, test_size=0.25, random_state=42, stratify=y)
models = {
    "Bagged decision trees": BaggingClassifier(estimator=DecisionTreeClassifier(),
n_estimators=100, random_state=42),
    "Random forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "Extra trees": ExtraTreesClassifier(n_estimators=100, random_state=42),
}
for name, model in models.items():
    model.fit(x_train, y_train)
    print(name, accuracy_score(y_test, model.predict(x_test)))
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 21: Ensemble Classification (Bagging, Random Forest, Extra Trees)

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p21.py
Bagged decision trees 0.7297297297297297
Random forest 0.7567567567567568
Extra trees 0.7747747747747747

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> 
```

Practical Exercise P22: Artificial Neural Network Backpropagation for XOR Logic Gate [CO5/BTL6]

1. Problem Statement (from Lab Manual):

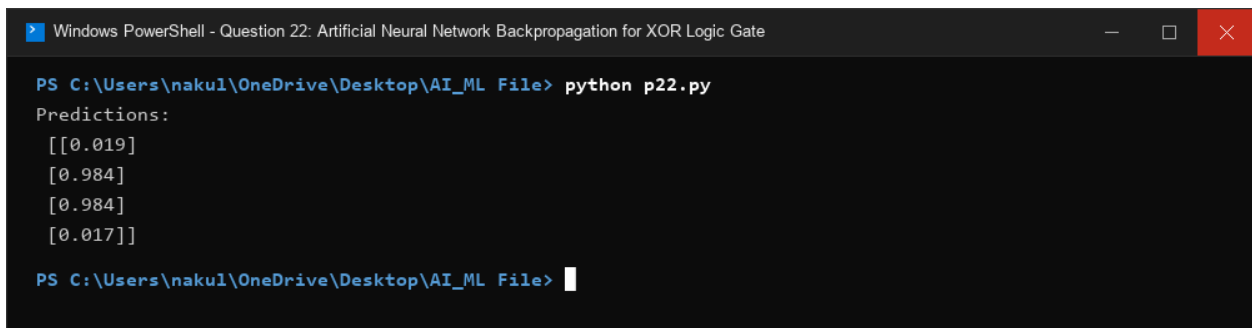
P22	Implement a simple neural network with: - 2 inputs - a hidden layer with 2 neurons (h1, h2) - an output layer with 1 neuron (o1) [CO5/BTL6]
-----	---

2. Python Source Code (p22.py):

```
import numpy as np

inputs = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
targets = np.array([[0], [1], [1], [0]], dtype=float) # XOR
rng = np.random.default_rng(1)
weights_hidden = rng.normal(size=(2, 2))
weights_output = rng.normal(size=(2, 1))
bias_hidden = np.zeros((1, 2))
bias_output = np.zeros((1, 1))
learning_rate = 0.5
sigmoid = lambda value: 1 / (1 + np.exp(-value))
for _ in range(10000):
    hidden = sigmoid(inputs @ weights_hidden + bias_hidden)
    output = sigmoid(hidden @ weights_output + bias_output)
    output_error = targets - output
    output_delta = output_error * output * (1 - output)
    hidden_delta = (output_delta @ weights_output.T) * hidden * (1 - hidden)
    weights_output += learning_rate * hidden.T @ output_delta
    weights_hidden += learning_rate * inputs.T @ hidden_delta
    bias_output += learning_rate * output_delta.sum(axis=0, keepdims=True)
    bias_hidden += learning_rate * hidden_delta.sum(axis=0, keepdims=True)
print("Predictions:\n", np.round(output, 3))
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 22: Artificial Neural Network Backpropagation for XOR Logic Gate

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p22.py
Predictions:
[[0.019]
 [0.984]
 [0.984]
 [0.017]]

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> 
```

Practical Exercise P23: Popularity-Based Movie Recommendation System

[CO6/BTL6]

1. Problem Statement (from Lab Manual):

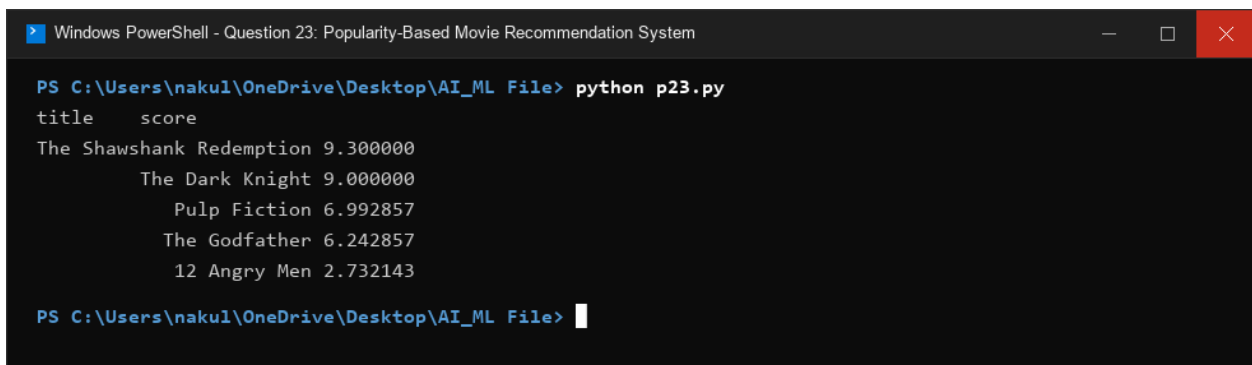
P23	Build a simplified clone of IMDB Top 250 Movies using metadata collected from IMDB. The following are the steps involved: 1. Decide on the metric or score to rate movies on. 2. Calculate the score for every movie. 3. Sort the movies based on the score and output the top results. 4. Use the Full Movie Lens dataset. [CO6/BTL6]
-----	--

2. Python Source Code (p23.py):

```
import pandas as pd

movies = pd.DataFrame({
    "title": ["The Shawshank Redemption", "The Godfather", "The Dark Knight", "12 Angry Men", "Pulp Fiction"],
    "rating": [9.3, 9.2, 9.0, 9.0, 8.9], "votes": [2800000, 1900000, 2800000, 850000, 2200000]})
movies["score"] = movies.rating * movies.votes / movies.votes.max()
print(movies.sort_values("score", ascending=False)[["title", "score"]].to_string(index=False))
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 23: Popularity-Based Movie Recommendation System

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p23.py
title      score
The Shawshank Redemption 9.300000
The Dark Knight 9.000000
Pulp Fiction 6.992857
The Godfather 6.242857
12 Angry Men 2.732143

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> 
```

Practical Exercise P24: Content-Based Recommendation using TF-IDF & Cosine Similarity [CO6/BTL6]

1. Problem Statement (from Lab Manual):

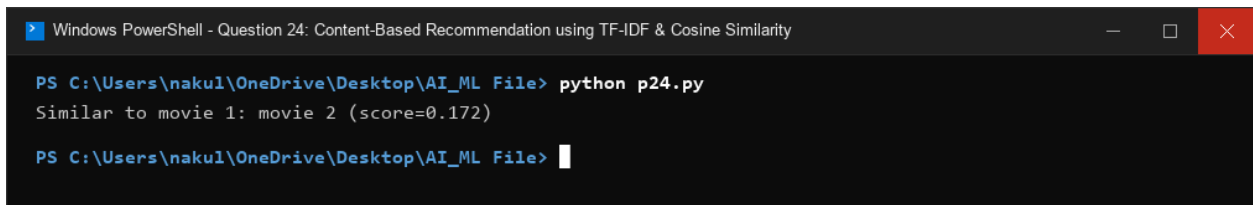
P24	Build a system that recommends movies that are similar to a particular movie. Compute pairwise cosine similarity scores for all movies based on their plot descriptions and recommend movies based on that similarity score threshold. The plot description is available to you as the overview feature in your metadata dataset. [CO6/BTL6]
-----	--

2. Python Source Code (p24.py):

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

movies = ["The space mission saves Earth", "A space crew explores a planet", "A detective solves a city crime", "A family protects their home"]
matrix = TfidfVectorizer(stop_words="english").fit_transform(movies)
similarity = cosine_similarity(matrix)
for index, score in enumerate(similarity[0]):
    if index != 0 and score >= 0.1:
        print(f"Similar to movie 1: movie {index + 1} (score={score:.3f})")
```

3. Execution Output / Result Screenshot:



```
Windows PowerShell - Question 24: Content-Based Recommendation using TF-IDF & Cosine Similarity

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p24.py
Similar to movie 1: movie 2 (score=0.172)

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>
```

Practical Exercise P25: Content-Based Recommendation using CountVectorizer & Cosine Similarity [CO3, CO6/BTL5]

1. Problem Statement (from Lab Manual):

P25	Build a recommender system based on the following metadata: the 3 top actors, the director, related genres, and the movie plot keywords.
	Note-The keywords, cast, and crew data are not available in your current dataset, so the first step would be to load and merge them into your main DataFrame metadata. [CO3, CO6/BTL5]

2. Python Source Code (p25.py):

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.metrics.pairwise import cosine_similarity

movies = [
    {"title": "Inception", "director": "Christopher Nolan", "actors": "Leonardo DiCaprio Joseph Gordon-Levitt", "genres": "action science fiction", "keywords": "dream heist"},
    {"title": "Interstellar", "director": "Christopher Nolan", "actors": "Matthew McConaughey Anne Hathaway", "genres": "science fiction drama", "keywords": "space time"},
    {"title": "The Matrix", "director": "Lana Wachowski", "actors": "Keanu Reeves Laurence Fishburne", "genres": "action science fiction", "keywords": "virtual reality"},
    {"title": "The Godfather", "director": "Francis Ford Coppola", "actors": "Marlon Brando Al Pacino", "genres": "crime drama", "keywords": "mafia family"},
]
documents = [" ".join(movie.values()) for movie in movies]
matrix = CountVectorizer().fit_transform(documents)
scores = cosine_similarity(matrix[0:1], matrix).ravel()
for index in scores.argsort()[::-1][1:4]:
    print(movies[index]["title"], round(scores[index], 3))
```

3. Execution Output / Result Screenshot:

```
Windows PowerShell - Question 25: Content-Based Recommendation using CountVectorizer & Cosine Similarity

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File> python p25.py
Interstellar 0.32
The Matrix 0.231
The Godfather 0.0

PS C:\Users\nakul\OneDrive\Desktop\AI_ML File>
```